

HANDS-ON #09

Auditoria de Dados com Triggers e Tabelas Temporais

Série 2 — SQL Server | Ponto de Conhecimento Tecnológico

Documento técnico para revisão · Laboratório SIManagerDB · 05/11/2026

Oito exercícios concluídos e validados com resultados reais no SQL Server 2019 Express. Este documento reproduz a implementação e as evidências partilhadas durante a execução.

1. Introdução e objetivos

O laboratório explora dois mecanismos complementares: triggers DML para registar operações e respetivo contexto, e tabelas temporais para preservar automaticamente estados anteriores dos dados. As tabelas originais da SIManagerDB não foram alteradas.

Objetivos: implementar auditoria de INSERT, UPDATE e DELETE; validar operações em conjunto; ativar SYSTEM_VERSIONING; reconstruir estados com AS OF, BETWEEN e ALL; comparar alcance e limitações.

2. Ambiente e preparação

Componente	Resultado validado
SQL Server	2019 Express (64-bit), versão 15.0.2190.7
SIManagerDB	ONLINE; compatibilidade 150
Aplicacao / Incidente / PedidoServico	25 / 50 000 / 12 000 registos
Tabelas originais	NON_TEMPORAL_TABLE

Foram criadas dbo.H09_IncidenteAuditoria (6 colunas, inicialmente vazia) e dbo.H09_LogAuditoria (7 colunas, inicialmente vazia). Na validação inicial foi corrigido o alias reservado Identity para EhIdentity.

```
USE SIManagerDB;
GO
IF OBJECT_ID('dbo.H09_IncidenteAuditoria','U') IS NULL
BEGIN
 CREATE TABLE dbo.H09_IncidenteAuditoria (
  IncidenteId INT IDENTITY(1,1) PRIMARY KEY,
  AplicacaoId INT NOT NULL,
  Titulo NVARCHAR(200) NOT NULL,
  Estado NVARCHAR(30) NOT NULL,
  Prioridade NVARCHAR(20) NOT NULL,
  DataCriacao DATETIME2(0) NOT NULL DEFAULT SYSUTCDATETIME()
 );
END;
GO
IF OBJECT_ID('dbo.H09_LogAuditoria','U') IS NULL
BEGIN
 CREATE TABLE dbo.H09_LogAuditoria (
  AuditoriaId BIGINT IDENTITY(1,1) PRIMARY KEY,
  IncidenteId INT NOT NULL,
  Operacao VARCHAR(10) NOT NULL,
  DadosAntes NVARCHAR(MAX) NULL,
  DadosDepois NVARCHAR(MAX) NULL,
  UtilizadorSQL SYSNAME NOT NULL,
  DataOperacao DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME(),
  CONSTRAINT CK_H09_Log_Operacao
   CHECK (Operacao IN ('INSERT','UPDATE','DELETE'))
 );
END;
GO
```

3. Exercício 02 — Auditoria de INSERT

O trigger AFTER INSERT lê o conjunto lógico inserted e grava um documento JSON por incidente. O teste inseriu dois incidentes numa única instrução, gerando duas entradas de auditoria; a associação de AuditoriaId a IncidenteId não segue necessariamente a ordem dos VALUES.

```
CREATE OR ALTER TRIGGER dbo.trg_H09_AuditoriaInsert
```

```
ON dbo.H09_IncidenteAuditoria AFTER INSERT
AS
BEGIN
 SET NOCOUNT ON;
 INSERT INTO dbo.H09_LogAuditoria
  (IncidenteId,Operacao,DadosAntes,DadosDepois,UtilizadorSQL)
 SELECT i.IncidenteId,'INSERT',NULL,
  (SELECT i.IncidenteId,i.AplicacaoId,i.Titulo,
          i.Estado,i.Prioridade,i.DataCriacao
   FOR JSON PATH, WITHOUT_ARRAY_WRAPPER),
  ORIGINAL_LOGIN()
 FROM inserted AS i;
END;
GO
```

Evidência: incidentes 1 e 2 criados; AuditoriaId 1 correspondeu ao incidente 2 e AuditoriaId 2 ao incidente 1; DadosAntes = NULL; DadosDepois = JSON; login e data preenchidos.

4. Exercício 03 — Auditoria de UPDATE

O trigger relaciona inserted e deleted pela chave IncidenteId, preservando JSON do estado anterior e do novo estado.

```
CREATE OR ALTER TRIGGER dbo.trg_H09_AuditoriaUpdate
ON dbo.H09_IncidenteAuditoria AFTER UPDATE
AS
BEGIN
 SET NOCOUNT ON;
 INSERT INTO dbo.H09_LogAuditoria
  (IncidenteId,Operacao,DadosAntes,DadosDepois,UtilizadorSQL)
 SELECT i.IncidenteId,'UPDATE',
  (SELECT d.IncidenteId,d.AplicacaoId,d.Titulo,
          d.Estado,d.Prioridade,d.DataCriacao
   FOR JSON PATH, WITHOUT_ARRAY_WRAPPER),
  (SELECT i.IncidenteId,i.AplicacaoId,i.Titulo,
          i.Estado,i.Prioridade,i.DataCriacao
   FOR JSON PATH, WITHOUT_ARRAY_WRAPPER),
  ORIGINAL_LOGIN()
 FROM inserted AS i
 INNER JOIN deleted AS d ON d.IncidenteId=i.IncidenteId;
END;
GO
```

Evidência: AuditoriaId 3, IncidenteId 1, estado Aberto → Em resolução, prioridade Alta → Crítica, em 05/11/2026 16:22:18.085 (valor de DataOperacao apresentado na sessão).

5. Exercício 04 — Auditoria de DELETE

O trigger AFTER DELETE consulta deleted, grava os valores anteriores e deixa DadosDepois a NULL.

```
CREATE OR ALTER TRIGGER dbo.trg_H09_AuditoriaDelete
ON dbo.H09_IncidenteAuditoria AFTER DELETE
AS
BEGIN
 SET NOCOUNT ON;
 INSERT INTO dbo.H09_LogAuditoria
  (IncidenteId,Operacao,DadosAntes,DadosDepois,UtilizadorSQL)
 SELECT d.IncidenteId,'DELETE',
  (SELECT d.IncidenteId,d.AplicacaoId,d.Titulo,
          d.Estado,d.Prioridade,d.DataCriacao
   FOR JSON PATH, WITHOUT_ARRAY_WRAPPER),
  NULL,ORIGINAL_LOGIN()
 FROM deleted AS d;
END;
GO
```

Evidência: o incidente 2 foi eliminado; AuditoriaId 4 preservou título «Erro na integração de dados», estado «Em análise» e prioridade «Média», com DadosDepois = NULL.

6. Exercício 05 — Operações em conjunto

Foram inseridos os incidentes 3, 4 e 5 numa única instrução; a auditoria gerou os identificadores 7, 6 e 5, respetivamente. A atualização em conjunto criou as auditorias 10, 9 e 8. Por fim, a eliminação conjunta dos incidentes 4 e 5 criou as auditorias 12 e 11.

Incidente	Estado antes → depois	Prioridade antes → depois
3	Aberto → Em análise	Alta → Crítica
4	Aberto → Em análise	Média → Alta
5	Aberto → Em análise	Crítica → Crítica

```
INSERT INTO dbo.H09_IncidenteAuditoria
 (AplicacaoId,Titulo,Estado,Prioridade)
VALUES
 (3,N'Falha no processamento de pedidos',N'Aberto',N'Alta'),
 (4,N'Erro de sincronização de dados',N'Aberto',N'Média'),
 (5,N'Indisponibilidade de serviço',N'Aberto',N'Crítica');
GO
UPDATE dbo.H09_IncidenteAuditoria
SET Estado=N'Em análise',
    Prioridade=CASE IncidenteId
      WHEN 3 THEN N'Crítica'
      WHEN 4 THEN N'Alta'
      WHEN 5 THEN N'Crítica' END
WHERE IncidenteId IN (3,4,5);
GO
DELETE FROM dbo.H09_IncidenteAuditoria
WHERE IncidenteId IN (4,5);
GO
```

Resultado final da tabela de laboratório de triggers: incidentes 1 e 3 preservados. O trigger regista linhas afetadas pelo UPDATE, ainda que um atributo individual mantenha o mesmo valor.

7. Exercício 06 — Tabelas temporais

Criou-se uma tabela temporal com colunas de período geridas pelo SQL Server e tabela de histórico dedicada. Foram realizados INSERT e dois UPDATE, obtendo três versões do incidente 1.

```
IF OBJECT_ID('dbo.H09_IncidenteTemporal','U') IS NULL
BEGIN
 CREATE TABLE dbo.H09_IncidenteTemporal (
  IncidenteId INT IDENTITY(1,1) PRIMARY KEY,
  Titulo NVARCHAR(200) NOT NULL,
  Estado NVARCHAR(30) NOT NULL,
  Prioridade NVARCHAR(20) NOT NULL,
  ValidoDesde DATETIME2(7)
   GENERATED ALWAYS AS ROW START NOT NULL,
  ValidoAte DATETIME2(7)
   GENERATED ALWAYS AS ROW END NOT NULL,
  PERIOD FOR SYSTEM_TIME (ValidoDesde,ValidoAte)
 ) WITH (SYSTEM_VERSIONING=ON
  (HISTORY_TABLE=dbo.H09_IncidenteTemporalHistorico));
END;
GO
INSERT INTO dbo.H09_IncidenteTemporal
 (Titulo,Estado,Prioridade)
```

```
VALUES (N'Falha de comunicação com serviço externo',
        N'Aberto',N'Alta');
GO
UPDATE dbo.H09_IncidenteTemporal
SET Estado=N'Em análise'
WHERE IncidenteId=1 AND Estado=N'Aberto';
GO
UPDATE dbo.H09_IncidenteTemporal
SET Estado=N'Resolvido',Prioridade=N'Baixa'
WHERE IncidenteId=1 AND Estado=N'Em análise';
GO
```

Estado	Prioridade	ValidoDesde (UTC)	ValidoAte (UTC)
Aberto	Alta	16:28:04.5099984	16:28:04.7141499
Em análise	Alta	16:28:04.7141499	16:28:59.2737395
Resolvido	Baixa	16:28:59.2737395	9999-12-31 (aberto)

Os instantes são os valores das colunas temporais apresentados no laboratório em 05/11/2026. São instantes UTC, associados ao início das transações de modificação, não carimbos de confirmação.

8. Exercício 07 — Consultas temporais

```
-- Estado num instante: Em análise / Alta
SELECT IncidenteId,Estado,Prioridade,ValidoDesde,ValidoAte
FROM dbo.H09_IncidenteTemporal
FOR SYSTEM_TIME AS OF '2026-11-05T16:28:30'
WHERE IncidenteId=1;
GO
-- Intervalo: Aberto e Em análise
SELECT IncidenteId,Estado,Prioridade,ValidoDesde,ValidoAte
FROM dbo.H09_IncidenteTemporal
FOR SYSTEM_TIME BETWEEN
 '2026-11-05T16:28:04.6000000'
 AND '2026-11-05T16:28:30.0000000'
WHERE IncidenteId=1 ORDER BY ValidoDesde;
GO
-- Todas as versões: 2 históricas e 1 atual
SELECT ROW_NUMBER() OVER
 (PARTITION BY IncidenteId ORDER BY ValidoDesde) AS NumeroVersao,
 IncidenteId,Estado,Prioridade,ValidoDesde,ValidoAte,
 CASE WHEN ValidoAte='9999-12-31T23:59:59.9999999'
      THEN 'ATUAL' ELSE 'HISTORICO' END AS TipoVersao
FROM dbo.H09_IncidenteTemporal
FOR SYSTEM_TIME ALL
WHERE IncidenteId=1 ORDER BY ValidoDesde;
GO
```

Resultados validados: AS OF devolveu «Em análise / Alta»; BETWEEN devolveu «Aberto / Alta» e «Em análise / Alta»; ALL devolveu as três versões na ordem correta.

9. Exercício 08 — Consolidação e comparação

Critério	Triggers DML	Tabelas temporais
Objeto auditado	Operações INSERT/UPDATE/DELETE	Versões dos dados
Valores anteriores/novos	JSON programado	Histórico automático
Login SQL	ORIGINAL_LOGIN() no exemplo	Não registado automaticamente
Consulta num instante	Requer lógica personalizada	FOR SYSTEM_TIME AS OF
Rollback	Auditoria na mesma transação	Histórico também transacional
Gestão	Código e estrutura próprios	Motor SQL Server + retenção/gestão

Validação final: 3 triggers ativos; 5 auditorias INSERT, 4 UPDATE, 3 DELETE (12 no total); tabela temporal ativa e 3 versões do incidente 1. Todos os oito exercícios foram validados com resultados enviados durante o laboratório.

10. Limitações, boas práticas e conclusões

1. ORIGINAL_LOGIN() identifica o login que iniciou a sessão, não necessariamente o utilizador final da aplicação. Se for necessário contexto de negócio, considerar SESSION_CONTEXT com controlos de confiança e rastreabilidade.

2. O histórico temporal preserva estados de linhas, mas não constitui por si só uma trilha de auditoria de ações, nem identifica o responsável. As versões são geridas em UTC e refletem instantes de início das transações.

3. Os triggers AFTER executam na mesma transação que a operação principal; um ROLLBACK remove também a respetiva entrada de auditoria. Não são, isoladamente, um registo imutável de tentativas falhadas.

4. Planear permissões, acesso ao histórico, proteção contra alterações administrativas, crescimento de armazenamento, retenção e eventuais necessidades de conformidade. A auditoria por JSON é uma escolha didática, não uma política universal de produção.

5. Para auditoria regulamentar ou investigação de segurança, avaliar mecanismos adicionais, incluindo SQL Server Audit e armazenamento protegido, conforme os requisitos.

Conclusão: a combinação de triggers DML e tabelas temporais oferece uma demonstração prática da diferença entre registar operações e reconstruir estados históricos. Os mecanismos podem ser complementares, desde que as garantias e limitações de cada um sejam explicitamente definidas.

Estado do documento: versão para revisão de conteúdo antes da preparação dos scripts finais, README e carrossel LinkedIn.